DynMIRTO — Blend Explorer (standalone)

A self-contained copy of the blend-optimization logic from caxelrud/DynMIRTO, inlined here so this single file has no dependency on the rest of the repo — only registered packages (JuMP, HiGHS, PlutoUI). That means it can run via Pluto's built-in "Edit or run this notebook" → Binder button, including from a phone browser, with no local Julia install.

This mirrors `src/Types.jl`, `src/BlendIndices.jl`, and `src/Optimizer.jl` in the repo as of when this notebook was last updated — it is not auto-synced, so re-copy from `src/` if those change. For local development against the live package, use `notebooks/blend_explorer.jl` instead.

Move the sliders below to change component economics/availability or product specs; the optimal blend recipe recomputes live.

```
md"""
# DynMIRTO — Blend Explorer (standalone)

A self-contained copy of the blend-optimization logic from
[caxelrud/DynMIRTO](https://github.com/caxelrud/DynMIRTO), inlined here so
this single file has no dependency on the rest of the repo — only
registered packages (JuMP, HiGHS, PlutoUI). That means it can run via
Pluto's built-in "Edit or run this notebook" → Binder button, including
from a phone browser, with no local Julia install.

This mirrors `src/Types.jl`, `src/BlendIndices.jl`, and `src/Optimizer.jl`
in the repo as of when this notebook was last updated — it is not
auto-synced, so re-copy from `src/` if those change. For local
development against the live package, use `notebooks/blend_explorer.jl`
instead.

Move the sliders below to change component economics/availability or
product specs; the optimal blend recipe recomputes live.
"""
```

```
begin
    using JuMP
    using HiGHS
    using PlutoUI
end
```

```
begin
    struct PropertySpec
        min::Union{Float64,Nothing}
        max::Union{Float64,Nothing}
        blend_rule::Symbol

        function PropertySpec(; min=nothing, max=nothing, blend_rule::Symbol=:linear)
            blend_rule in (:linear, :index) ||
                error("blend_rule must be :linear or :index, got :$blend_rule")
            min === nothing || max === nothing || min <= max ||
                error("PropertySpec min ($min) must be <= max ($max)")
            new(min, max, blend_rule)
        end
    end

    struct Component
        id::String
        name::String
        cost::Float64
        available::Float64
        min_available::Float64
        properties::Dict{Symbol,Float64}

        function Component(id, name, cost, available; min_available=0.0,
        properties=Dict{Symbol,Float64}())
            available >= min_available ||
                error("Component $id: available ($available) must be >= min_available
                ($min_available)")
            new(id, name, Float64(cost), Float64(available), Float64(min_available),
            properties)
        end
    end

    struct Product
        id::String
        name::String
        demand::Float64
        price::Float64
        specs::Dict{Symbol,PropertySpec}
        eligible_components::Union{Vector{String},Nothing}

        function Product(id, name, demand; price=0.0, specs=Dict{Symbol,PropertySpec}
        (), eligible_components=nothing)
            demand >= 0 || error("Product $id: demand must be >= 0")
            new(id, name, Float64(demand), Float64(price), specs, eligible_components)
        end
    end

    is_eligible(c::Component, p::Product) =
        p.eligible_components === nothing || c.id in p.eligible_components

    struct BlendRecipe
        product_id::String
        volumes::Dict{String,Float64}
        fractions::Dict{String,Float64}
        resulting_properties::Dict{Symbol,Float64}
        cost::Float64
    end
end
```

```
begin
    const _INDEX_REGISTRY = Dict{Symbol,NamedTuple{(:to_index,
    :from_index),Tuple{Function,Function}}}()

    function register_blend_index!(property::Symbol, to_index::Function,
    from_index::Function)
        _INDEX_REGISTRY[property] = (to_index=to_index, from_index=from_index)
        return nothing
    end

    has_blend_index(property::Symbol) = haskey(_INDEX_REGISTRY, property)

    function to_index_space(property::Symbol, value::Float64)
        has_blend_index(property) ||
            error("No blending index registered for property :$property.")
        _INDEX_REGISTRY[property].to_index(value)
    end

    function from_index_space(property::Symbol, value::Float64)
        has_blend_index(property) ||
            error("No blending index registered for property :$property.")
        _INDEX_REGISTRY[property].from_index(value)
    end

    # RVP: power-law approximation 'BI = RVP^1.25', a commonly cited
    # approximation for RVP's non-linear blending behavior.
    register_blend_index!(:RVP, rvp -> rvp^1.25, bi -> bi^(1 / 1.25))

    # RON / MON: real octane blending indices are refinery- and
    # crude-slate-specific. This exponent is an ILLUSTRATIVE placeholder
    # demonstrating the index mechanism — not a calibrated correlation.
    register_blend_index!(:RON, ron -> ron^1.06, bi -> bi^(1 / 1.06))
    register_blend_index!(:MON, mon -> mon^1.06, bi -> bi^(1 / 1.06))
end

```

```
optimize_blend (generic function with 1 method)
```

```
begin
    function _to_solver_space(rule::Symbol, prop::Symbol, value::Float64)
        rule === :linear && return value
        rule === :index && return to_index_space(prop, value)
        error("Unknown blend_rule :$rule")
    end

    function _from_solver_space(rule::Symbol, prop::Symbol, value::Float64)
        rule === :linear && return value
        rule === :index && return from_index_space(prop, value)
        error("Unknown blend_rule :$rule")
    end

    struct OptimizationResult
        status::Symbol
        recipes::Dict{String,BlendRecipe}
        diagnostics::Vector{String}
    end

    function _prevalidate(components::Vector{Component}, products::Vector{Product},
    pairs::Vector{Tuple{String,String}})
        notes = String[]
        comp_by_id = Dict(c.id => c for c in components)

        for p in products
            eligible_ids = [cid for (cid, pid) in pairs if pid == p.id]
            if isempty(eligible_ids)
                push!(notes, "Product $(p.id): no eligible components at all.")
                continue
            end
            total_available = sum(comp_by_id[cid].available for cid in eligible_ids)
            if total_available < p.demand
                push!(notes,
                    "Product $(p.id): demand ($(p.demand)) exceeds total available
                    volume " *
                    "of eligible components ($(total_available)).")
            end
            total_floor = sum(comp_by_id[cid].min_available for cid in eligible_ids)
            if total_floor > p.demand
                push!(notes,
                    "Product $(p.id): sum of component must-use minimums
                    ($(total_floor)) " *
                    "exceeds demand ($(p.demand)).")
            end
            for (prop, spec) in p.specs
                vals = [comp_by_id[cid].properties[prop] for cid in eligible_ids if
                haskey(comp_by_id[cid].properties, prop)]
                isempty(vals) && continue
                lo, hi = extrema(vals)
                if spec.max !== nothing && lo > spec.max
                    push!(notes,
                        → "Product $(p.id), property :$prop: every eligible component " *
                        → "($(lo)-$(hi)) exceeds the max spec ($(spec.max)); no blend can
                        meet it.")
                end
                if spec.min !== nothing && hi < spec.min
                    push!(notes,
                        → "Product $(p.id), property :$prop: every eligible component " *
                        → "($(lo)-$(hi)) is below the min spec ($(spec.min)); no blend can
                        meet it.")
                end
            end
        end

        return notes
    end

    function optimize_blend(components::Vector{Component}, products::Vector{Product};
        optimizer=HiGHS.Optimizer)

        isempty(components) && error("optimize_blend: no components given")
        isempty(products) && error("optimize_blend: no products given")

        pairs = Tuple{String,String}[]
        for p in products, c in components
            is_eligible(c, p) && push!(pairs, (c.id, p.id))
        end

        diagnostics = _prevalidate(components, products, pairs)
        if !isempty(diagnostics)
            return OptimizationResult(:infeasible, Dict{String,BlendRecipe}(),
            diagnostics)
        end

        model = Model(optimizer)
        set_silent(model)

        x = Dict{Tuple{String,String},JuMP.VariableRef}()
        for (cid, pid) in pairs
            x[(cid, pid)] = @variable(model, lower_bound = 0, base_name =
            "x_$(cid)_$(pid)")
        end

        comp_by_id = Dict(c.id => c for c in components)

        for p in products
            vars = [x[(c.id, p.id)] for c in components if haskey(x, (c.id, p.id))]
            isempty(vars) && continue
            @constraint(model, sum(vars) == p.demand)
        end

        for c in components
            vars = [x[(c.id, p.id)] for p in products if haskey(x, (c.id, p.id))]
            isempty(vars) && continue
            @constraint(model, sum(vars) <= c.available)
            if c.min_available > 0
                @constraint(model, sum(vars) >= c.min_available)
            end
        end

        for p in products, (prop, spec) in p.specs
            vars_vals = Tuple{JuMP.VariableRef,Float64}[]
            for c in components
                haskey(x, (c.id, p.id)) || continue
                haskey(c.properties, prop) ||
                    error("Component $(c.id) is eligible for product $(p.id) but has no
                    value for property :$prop")
                val = _to_solver_space(spec.blend_rule, prop, c.properties[prop])
                push!(vars_vals, (x[(c.id, p.id)], val))
            end
            isempty(vars_vals) && continue
            weighted = sum(v * val for (v, val) in vars_vals)
            if spec.min !== nothing
                lo = _to_solver_space(spec.blend_rule, prop, spec.min)
                @constraint(model, weighted >= lo * p.demand)
            end
            if spec.max !== nothing
                hi = _to_solver_space(spec.blend_rule, prop, spec.max)
                @constraint(model, weighted <= hi * p.demand)
            end
```

```
        end

        @objective(model, Min, sum(comp_by_id[cid].cost * v for ((cid, _), v) in x))

        optimize!(model)
        status = termination_status(model)

        if status != MOI.OPTIMAL
            note = "Solver returned $(status). This can mean the specs/availability are
            " *
                "infeasible together, or (rarely) numerical trouble."
            return OptimizationResult(:infeasible, Dict{String,BlendRecipe}(), [note])
        end

        recipes = Dict{String,BlendRecipe}()
        for p in products
            volumes = Dict{String,Float64}()
            for c in components
                haskey(x, (c.id, p.id)) || continue
                v = value(x[(c.id, p.id)])
                v > 1e-9 && (volumes[c.id] = v)
            end
            fractions = Dict(cid => v / p.demand for (cid, v) in volumes)
            cost = sum(comp_by_id[cid].cost * v for (cid, v) in volumes; init=0.0)

            resulting = Dict{Symbol,Float64}()
            all_props = Set{Symbol}()
            for cid in keys(volumes)
                union!(all_props, keys(comp_by_id[cid].properties))
            end
            for prop in all_props
                rule = haskey(p.specs, prop) ? p.specs[prop].blend_rule : :linear
                weighted = sum(_to_solver_space(rule, prop,
                comp_by_id[cid].properties[prop]) * v
                    →→→ for (cid, v) in volumes if haskey(comp_by_id[cid].properties,
                    prop); init=0.0)
                resulting[prop] = _from_solver_space(rule, prop, weighted / p.demand)
            end

            recipes[p.id] = BlendRecipe(p.id, volumes, fractions, resulting, cost)
        end

        return OptimizationResult(:optimal, recipes, String[])
    end
end
```

▸ Product("REG", "Regular Unleaded", 1000.0, 0.0, Dict(:RON ⇒ PropertySpec(87.0, nothing, :inde

```
begin
    base_components = [
        Component("BUT", "Butane", 45.0, 200.0; properties=Dict(:RON=>92.0, :RVP=>52.0,
        :sulfur_ppm=>0.0, :benzene_pct=>0.0)),
        Component("REF", "Reformate", 58.0, 3000.0; properties=Dict(:RON=>98.0,
        :RVP=>3.0, :sulfur_ppm=>5.0, :benzene_pct=>1.8)),
        Component("FCC", "FCC Gasoline", 52.0, 3000.0; properties=Dict(:RON=>91.0,
        :RVP=>5.0, :sulfur_ppm=>300.0, :benzene_pct=>0.5)),
        Component("ALK", "Alkylate", 60.0, 2000.0; properties=Dict(:RON=>96.0,
        :RVP=>5.0, :sulfur_ppm=>1.0, :benzene_pct=>0.0)),
    ]
    base_product = Product("REG", "Regular Unleaded", 1000.0;
        specs=Dict(
            :RON => PropertySpec(; min=87.0, blend_rule=:index),
            :RVP => PropertySpec(; max=9.0, blend_rule=:index),
            :sulfur_ppm => PropertySpec(; max=80.0, blend_rule=:linear),
            :benzene_pct => PropertySpec(; max=1.3, blend_rule=:linear),
        ))
end
```

Controls

```
md"""## Controls"""
```

Reformate cost ($ per volume unit)

```
md"**Reformate cost** (\$ per volume unit)"
```

58.0

```
@bind ref_cost Slider(40.0:1.0:80.0; default=58.0, show_value=true)
```

FCC gasoline available (volume units)

```
md"**FCC gasoline available** (volume units)"
```

3000.0

```
@bind fcc_available Slider(500.0:100.0:4000.0; default=3000.0, show_value=true)
```

Regular unleaded — minimum RON

```
md"**Regular unleaded — minimum RON**"
```

87.0

```
@bind ron_min Slider(85.0:0.5:95.0; default=87.0, show_value=true)
```

Regular unleaded — maximum sulfur (ppm)

```
md"**Regular unleaded — maximum sulfur (ppm)**"
```

80.0

```
@bind sulfur_max Slider(10.0:5.0:150.0; default=80.0, show_value=true)
```

▸ OptimizationResult(:optimal, Dict("REG" => BlendRecipe("REG", Dict(⋯ more), Dict(⋯ more), Dict(

```
begin
    function override_component(c)
        c.id == "REF" && return Component(c.id, c.name, ref_cost, c.available;
        min_available=c.min_available, properties=c.properties)
        c.id == "FCC" && return Component(c.id, c.name, c.cost, fcc_available;
        min_available=c.min_available, properties=c.properties)
        return c
    end
    components = override_component.(base_components)
    product = Product(base_product.id, base_product.name, base_product.demand;
        price=base_product.price,
        specs=merge(base_product.specs, Dict(
            :RON => PropertySpec(; min=ron_min, blend_rule=:index),
            :sulfur_ppm => PropertySpec(; max=sulfur_max, blend_rule=:linear),
        )),
        eligible_components=base_product.eligible_components)
    result = optimize_blend(components, [product])
end
```

Current component slate

id	name	cost	available	RON	RVP	sulfur_ppm	benzene_pct
BUT	Butane	45.0	200.0	92.0	52.0	0.0	0.0
REF	Reformate	58.0	3000.0	98.0	3.0	5.0	1.8
FCC	FCC Gasoline	52.0	3000.0	91.0	5.0	300.0	0.5
ALK	Alkylate	60.0	2000.0	96.0	5.0	1.0	0.0

```
begin
    comp_rows = join([
        "<tr><td>$(c.id)</td><td>$(c.name)</td><td>$(round(c.cost;digits=2))</td>
        <td>$(round(c.available;digits=1))</td>" *
        "<td>$(get(c.properties,:RON,"-"))</td><td>$(get(c.properties,:RVP,"-"))</td>" *
        "<td>$(get(c.properties,:sulfur_ppm,"-"))</td>
        <td>$(get(c.properties,:benzene_pct,"-"))</td></tr>"
        for c in components
    ])
    HTML("""
    <h4>Current component slate</h4>
    <table>
    <tr><th>id</th><th>name</th><th>cost</th><th>available</th><th>RON</th><th>RVP</th>
    <th>sulfur_ppm</th><th>benzene_pct</th></tr>
    $comp_rows
    </table>
    """)
end
```

Recipe (cost = 55470.08)

ALK: 14.3 (1.4%)

BUT: 78.8 (7.9%)

FCC: 255.8 (25.6%)

REF: 651.2 (65.1%)

```
if result.status == :optimal
    recipe = result.recipes[product.id]
    bars = join([
        """<div style="margin:6px 0;">
            <div style="font-family:monospace;font-size:0.9em;">$(cid): $(round(vol;
            digits=1)) ($(round(100*recipe.fractions[cid]; digits=1))%)</div>
            <div style="background:#e3e3e3;border-
            radius:3px;overflow:hidden;width:320px;">
                <div
                style="width:$(100*recipe.fractions[cid])%;background:#4c78a8;height:14p
                x;"></div>
            </div>
        </div>"""
        for (cid, vol) in sort(collect(recipe.volumes); by=first)
    ])
    HTML("<h4>Recipe (cost = $(round(recipe.cost; digits=2)))</h4>$bars")
else
    diag_items = join(["<li>$d</li>" for d in result.diagnostics])
    HTML("<h4 style=\"color:#b00;\">Infeasible</h4><ul>$diag_items</ul>")
end
```

Resulting quality vs. spec

Property	Value	Min	Max
RON	95.712	87.0	-
RVP	9.0	-	9.0
benzene_pct	1.3	-	1.3
sulfur_ppm	80.0	-	80.0

```
if result.status == :optimal
    recipe2 = result.recipes[product.id]
    spec_rows = join([
        "<tr><td>$(prop)</td><td>$(round(get(recipe2.resulting_properties, prop, NaN);
        digits=3))</td>" *
        "<td>$(spec.min === nothing ? "-" : spec.min)</td><td>$(spec.max === nothing ?
        "-" : spec.max)</td></tr>"
        for (prop, spec) in sort(collect(product.specs); by=first)
    ])
    HTML("""
    <h4>Resulting quality vs. spec</h4>
    <table>
    <tr><th>Property</th><th>Value</th><th>Min</th><th>Max</th></tr>
    $spec_rows
    </table>
    """)
else
    HTML("<h4>Resulting quality vs. spec</h4><p>Scenario is infeasible with the current
    sliders — see diagnostics above.</p>")
end
```